

Persistent Homology: A Pedagogical Introduction with Biological Applications

Aurelie Jodelle Kemme^{a,b,*}, Collins A. Agyingi^{a,**}

^a*Department of Mathematical Sciences, University of South Africa, Pretoria, South Africa*

^b*African Institute for Mathematical Sciences, Research and Innovation Centre, Kigali, Rwanda*

Abstract

Persistent Homology (PH) is a fundamental tool in computational topology, designed to uncover the intrinsic geometric and topological features of data across multiple scales. Originating within the broader framework of Topological Data Analysis (TDA), PH has found diverse applications ranging from protein structure and knot analysis to financial domains such as Bitcoin behavior and stock market dynamics. Despite its growing relevance, there remains a lack of accessible resources that bridge the gap between theoretical foundations and practical implementation for beginners. This paper offers a clear and comprehensive introduction to persistent homology, guiding readers from core concepts to real-world application. Specifically, we illustrate the methodology through the analysis of a 3-1 supercoiled DNA structure. The paper is tailored for readers without prior exposure to algebraic topology, aiming to demystify persistent homology and foster its broader adoption in data analysis tasks.

1. Introduction

Persistent Homology (PH) is a powerful topological tool within the field of Topological Data Analysis (TDA), emerging from foundational works in applied algebraic topology and computational geometry [9, 30]. Developed to address limitations in traditional machine learning (ML) methods [5, 26], PH offers a means to capture the global geometric and topological structure of complex data by identifying and tracking features such as loops, cycles, and voids across multiple scales [2, 4, 27]. Conventional Machine Learning (ML) approaches—whether supervised, unsupervised, or reinforcement learning—primarily rely on statistical relationships, distances, or local density estimations within datasets. Supervised learning focuses on leveraging labelled data to predict outcomes [11, 19, 22], unsupervised learning seeks patterns and groupings without predefined labels [16, 20], and reinforcement learning trains agents to interact with environments to maximize rewards [15, 17]. While effective in many scenarios, these paradigms often struggle to reveal the deeper, intrinsic structure of high-dimensional or non-linear data. Persistent Homology complements unsupervised learning by shifting the focus from local patterns to the global topological invariants of data—properties that remain unchanged under continuous transformations [12]. Unlike standard clustering techniques, which typically depend on Euclidean distances or local density measures [10], PH constructs a sequence of nested simplicial complexes known as a filtration. Through this process, PH systematically captures the birth and death of topological features across scales, providing insights into the underlying shape and connectivity of data.

*Emails: akemme@aimsric.org/jkemme@quantumleapafrica.org

**Email: agyinca@unisa.ac.za

that traditional methods overlook. For instance, in clustering tasks, conventional algorithms may effectively group points based on proximity but fail to detect essential structures such as holes or voids, which could signify meaningful gaps or cycles within the data. Persistent Homology, by contrast, identifies these features by analyzing how connected components, loops, and higher-dimensional voids evolve as the scale parameter changes [23]. Visualization tools such as persistence barcodes and persistence diagrams further enhance interpretability, offering a clear representation of the lifespan of these topological features [8]. While PH begins with pairwise Euclidean distances to construct simplicial complexes (e.g., Vietoris–Rips complexes), its strength lies in transcending simple metric-based approaches, revealing multi-scale topological signatures that enrich data understanding far beyond what conventional ML techniques can provide. In this context, Persistent Homology stands out as an innovative and robust method, bridging gaps in classical data analysis by incorporating topological perspectives into the exploration of complex datasets.

1.1. Overview of the paper

This paper offers an accessible introduction to persistent homology, a central concept in topological data analysis, using intuitive examples built from point cloud data. Our goal is to guide the reader step by step from basic topological ideas to their powerful application in data analysis. We begin on the second section by laying the mathematical foundation, introducing key notions such as topological spaces, homeomorphisms, homotopy, and metric spaces that allow us to understand shapes beyond rigid geometry. This section also introduces the idea of a point cloud—a collection of data points in space—and the role of distance functions in measuring similarity between them. This sets the stage for constructing meaningful topological structures from raw data. In the Third section, we explore simplicial complexes and homology. Simplicial complexes form the building blocks used to approximate shapes in data. These combinatorial structures enable us to capture the geometry and connectivity of the data in a form that can be analyzed algebraically. Next, but still in the same section, we delve into chain complexes and homology, which allow us to detect topological features. This step transforms our geometric constructions into algebraic objects that can be computed. The fourth section introduces the core topic of the paper: persistent homology. Here, we show how topological features evolve as we vary the scale at which we view the data. Using techniques like filtration, barcodes, and persistence diagrams, we learn how to capture and visualize multi-scale structures in complex datasets. The fifth section presents a real-world implementation of persistent homology applied to the 3-1 supercoiled DNA structure. Finally, the last section presents the conclusion and some future research directions. This paper is intended for readers with little to no background in algebraic topology. Our focus is on clarity and intuition, supported by real-world examples to demonstrate how persistent homology can uncover hidden structure in data.

2. Mathematical Preliminaries

We familiarize the reader with mathematical concepts from topology and metric spaces, forming the backbone of persistent homology.

2.1. Topology

Topology is a branch of mathematics that focuses on the properties of shapes that stay the same when they are stretched, bent, or twisted—but not torn or glued. This means that two shapes are considered the same in topology if one can be smoothly transformed into the other without cutting or attaching new parts.

A classic example is the comparison between a coffee mug and a doughnut: although they look very different, both have exactly one hole. In the eyes of topology, this makes them equivalent because their essential structure—what mathematicians call the *topological genus*—is the same.



Figure 1: Some topologically equivalent objects. source: Reese Cabrera

The image in Figure 1 offers a fun and intuitive way to understand the concept of **genus** in topology. The **genus** of a shape is a count of how many "holes" or "handles" it has—more formally, it's the number of independent loops you can draw on the object without being able to shrink them down to a point.

The categories in the image are:

1. **Genus 0:** These objects have **no holes**. For example, a **marble** or a **bowling ball** is topologically equivalent to a sphere. You can't pass anything through them—they are "solid."
2. **Genus 1:** These shapes have **one hole**. The classic example is a **doughnut** (or torus), but interestingly, a **coffee cup** also fits here. Why? Because the handle of the cup forms a loop that is topologically the same as the hole in a doughnut.
3. **Genus 2:** Objects like **kettles** and **scissors** have **two holes**—think of them as having two handles or loops.
4. **Genus 3 or more:** Shapes such as a **strainer** or **grater** have **three or more holes**. These are more complex in their structure and can't be continuously deformed into simpler ones without "breaking" them.

This classification helps topologists understand which shapes are "the same" from a topological point of view, even if they look quite different geometrically. It's a key idea behind why a mug and a doughnut are topological twins!

Before we can understand how topology helps us study data using tools like persistent homology, we need to take a small step into the mathematical language of topology. Just as geometry is built on points, lines, and angles, topology is built on the idea of a **topological space**. A topological space provides a flexible framework to describe the notions of *closeness* and *connectedness* without

relying on precise measurements like distance. This allows us to study the *shape* or *structure* of objects in a very general way, focusing on properties that remain unchanged when we bend or stretch them. Let us now introduce this fundamental concept more formally:

Definition 1 (Topological Space). A **topological space** (X, τ) is a set X together with a collection of subsets τ , called a topology, satisfying the following properties:

1. $\emptyset \in \tau$ and $X \in \tau$;
2. τ is closed under finite intersections: if $A, B \in \tau$, then $A \cap B \in \tau$;
3. τ is closed under arbitrary unions: if $\{A_i\}_{i \in I} \subseteq \tau$, then $\bigcup_{i \in I} A_i \in \tau$.

The sets in τ are called open sets.

Once we have the notion of a *topological space*, we can begin to compare different shapes by asking whether they are *the same* from a topological point of view. But what does "the same" really mean in topology?

One of the strongest ways to consider two spaces as equivalent is through a concept called a **homeomorphism**, which is a continuous deformation between two spaces without cutting or gluing.

Definition 2 (Homeomorphism). Let X and Y be two topological spaces. A function $f : X \rightarrow Y$ is called a **homeomorphism** if:

1. f is bijective (one-to-one and onto),
2. f is continuous,
3. The inverse function $f^{-1} : Y \rightarrow X$ is also continuous.

If such a deformation exists, the two spaces are said to be *homeomorphic*—for example, the famous case where a coffee mug is homeomorphic to a doughnut because both have one hole. However, in many situations, this notion can be too strict. This is where the idea of **homotopy** comes into play. In topology and algebra, **homotopy** provides a more flexible, or *weaker*, notion of equivalence than homeomorphism. It allows us to classify shapes that may not be exactly deformable into each other but still share the same fundamental *topological features*.

Definition 3 (Homotopy). Let X and Y be topological spaces, and let $f, g : X \rightarrow Y$ be two continuous functions. A **homotopy** between f and g is a continuous function:

$$H : X \times [0, 1] \rightarrow Y$$

such that:

$$H(x, 0) = f(x) \quad \text{and} \quad H(x, 1) = g(x) \quad \text{for all } x \in X.$$

If such a function H exists, we say that f and g are **homotopic**, denoted by $f \simeq g$.

When two spaces can be continuously transformed into each other through a family of continuous maps, we say they are *homotopically equivalent*. Homotopy focuses less on precise shape and more on preserving key features like connectedness and the number of holes. For example, a solid disk and a single point are homotopically equivalent because the disk can be continuously "shrunk" down to a point without tearing it.

Definition 4 (Homotopy Equivalence). Two topological spaces X and Y are said to be **homotopy equivalent** if there exist continuous functions:

$$f : X \rightarrow Y \quad \text{and} \quad g : Y \rightarrow X$$

such that:

$$g \circ f \simeq id_X \quad \text{and} \quad f \circ g \simeq id_Y,$$

where id_X and id_Y are the identity maps on X and Y , respectively.

In this case, X and Y are considered to have the same "shape" from the perspective of homotopy theory, even if they are not homeomorphic.

This concept of homotopy is essential in understanding how topology abstracts away geometric details to focus on the underlying structure, which will be crucial as we move toward tools like **homology** and **persistent homology** in data analysis.



Figure 2: A coffee mug and a donut are homotopically equivalent. source: Marlowe Hood, GMA News

In real-world situations, data is rarely simple or neatly organized. Instead, it often forms complex patterns that are difficult to interpret using traditional methods. Understanding the overall structure—or *shape*—of such data can quickly become challenging. This is exactly where the field of **Topological Data Analysis** (TDA) proves invaluable.

TDA, a growing area within data science and artificial intelligence, emerged from advances in applied algebraic topology and computational geometry. Its main purpose is to study the *shape of data*, based on the idea that data isn't just a collection of points—it has structure, and this structure carries meaningful information. But how can we extract insights from the shape of data? To answer this, TDA offers several powerful tools, with two of the most well-known being the **Mapper algorithm** and **Persistent Homology (PH)**. The **Mapper algorithm** works by covering the dataset and connecting overlapping regions to form a simplified representation called a *simplicial complex*. Think of it as creating a network or skeleton that captures the main features of the data's shape. However, using Mapper requires careful choices, like selecting the right function to project the data and deciding how to cover it, which can influence the results significantly. **Persistent Homology**, on the other hand, takes a different and often more systematic approach. Instead of relying on user-defined functions and covers, PH starts with data represented as a **point cloud**—a set of points in space. It calculates the distances between these points (usually using Euclidean distance) and builds a series of increasingly connected shapes called *simplicial complexes*. This process, known as a **filtration**, allows us to observe how topological features like **connected components**, **loops**, and **voids** appear and disappear as we "zoom out" on the data. The beauty of this method lies in its ability to detect which features are significant and which are just noise. Thanks to concepts from algebraic topology—particularly **homology**—we can track these features across different scales. The number of features in each

dimension is counted using **Betti numbers**, giving us a clear summary of the data's shape. Finally, to make these insights more accessible, Persistent Homology provides visual tools such as **persistence barcodes** and **persistence diagrams**. These visuals show how long each topological feature "persists" throughout the filtration, helping us distinguish meaningful patterns from short-lived noise. It's important to note that the features we study belong to mathematical objects called **homology groups**, which, in this context, can be thought of as vector spaces where each vector represents a topological feature.

Before diving deeper into how Persistent Homology works, we must introduce two essential concepts that form the foundation of this method: the idea of a **point cloud** and the role of **metric spaces**. These will help us understand how to build a successful persistent homology pipeline.

2.2. Metric Spaces

Metric Spaces are a fundamental ideas in mathematics that formalizes what we intuitively think of as "distance". Before formally defining metric space and explaining its role in persistent homology computation, we first need to understand one fundamental concept: **point clouds**. Points cloud serves in general as the starting state for many techniques in Topological Data Analysis, including Persistent Homology.

2.2.1. Point Clouds

A **point cloud** is simply a collection of individual points scattered in space. You can think of it as a digital snapshot of an object or shape, where instead of smooth surfaces or lines, we only have dots representing key positions. Point clouds are often used to capture the structure of objects in three dimensions (3D), but they can exist in any number of dimensions depending on the data. Typically, each point in a point cloud is defined by coordinates—for example, (X, Y, Z) in a 3D space. These points might come from various sources: they could be generated by 3D scanners, simulations, or even randomly, depending on whether the underlying distribution is known or unknown. While point clouds give us a way to visualize shapes or patterns, it's important to note that they don't naturally include connections or order between points. They are just scattered data, which means they lack an inherent structure. This is where mathematics helps us make sense of them. Point clouds can take many forms, depending on how the data is distributed. Some common examples include:

- **Random Point Cloud:** Points are scattered without any clear pattern, often generated randomly.
- **Clusters:** Points group together in smaller regions, revealing natural groupings or categories within the data.
- **Spiral or Ellipsoid Shapes:** Points are arranged in a curved or circular pattern, suggesting more complex structures.

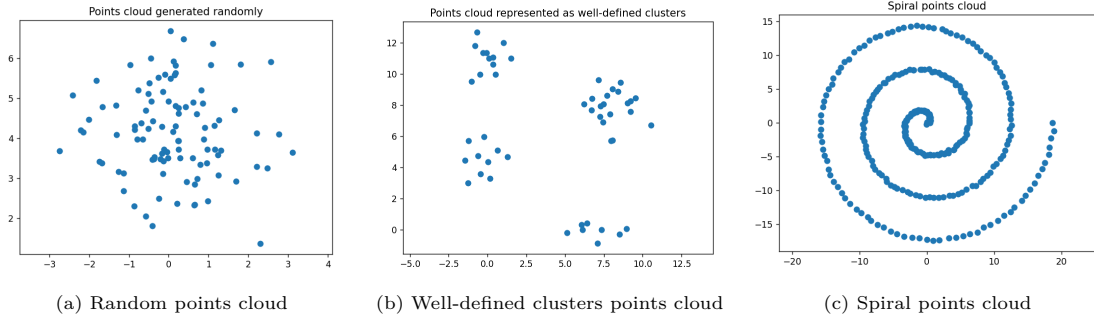


Figure 3: Visualization of three different types of point clouds.

To analyze point clouds, we need a way to measure how "close" or "far apart" the points are from each other. For this, we rely on the idea of a **metric space**, where the concept of distance is clearly defined. By knowing how to calculate distances between points, we can begin to detect patterns, shapes, and connections within the cloud. In the next section, we will introduce **metric spaces** more formally, to understand how distances allow us to explore and analyze these point clouds effectively.

2.2.2. Metric Spaces

To better understand how we measure distances between points in a point cloud, we need to define what we refer to as a **metric space**. At its core, a metric space is simply a set of points where a clear rule—called a *metric*—tells us how far apart any two points are. This notion of distance is essential for analyzing data, especially when we want to study shapes, patterns, or connections within a dataset.

Here is a simple definition:

Definition 5 (Metric Space). A **metric space** is a pair (M, d) where:

- M is a set of points,
- $d : M \times M \rightarrow \mathbb{R}$ is a function that assigns a distance between any two points in M .

The function d is called a **metric** if it satisfies the following four properties for all points $x, y, z \in M$:

1. **Identity:** $d(x, x) = 0$ (The distance from a point to itself is zero.)
2. **Positivity:** $d(x, y) > 0$ if $x \neq y$ (Distances are always positive between distinct points.)
3. **Symmetry:** $d(x, y) = d(y, x)$ (The distance from x to y is the same as from y to x .)
4. **Triangle Inequality:** $d(x, z) \leq d(x, y) + d(y, z)$ (The direct path is always the shortest.)

If these conditions are satisfied, we say that (M, d) is a metric space.

To make this more concrete, let's look at a familiar example.

Example 1 (Metric on the Real Line). Consider the set of real numbers $M = \mathbb{R}$. We can define the distance between any two points x and y as:

$$d(x, y) = |x - y|,$$

where $|\cdot|$ denotes the absolute value.

This is the usual way we measure distance on a number line. Let's briefly check that this function satisfies the four properties of a metric:

- **Identity:** For any $x \in \mathbb{R}$, we have $d(x, x) = |x - x| = 0$.
- **Positivity:** If $x \neq y$, then $|x - y| > 0$.
- **Symmetry:** $|x - y| = |y - x|$.
- **Triangle Inequality:** For any $x, y, z \in \mathbb{R}$,

$$|x - z| \leq |x - y| + |y - z|.$$

Since all four conditions are satisfied, $(\mathbb{R}, |\cdot|)$ is indeed a metric space.

Example 2 (Metric). Figure 4 illustrates various distance measures. In our case, we specifically use the Euclidean distance.

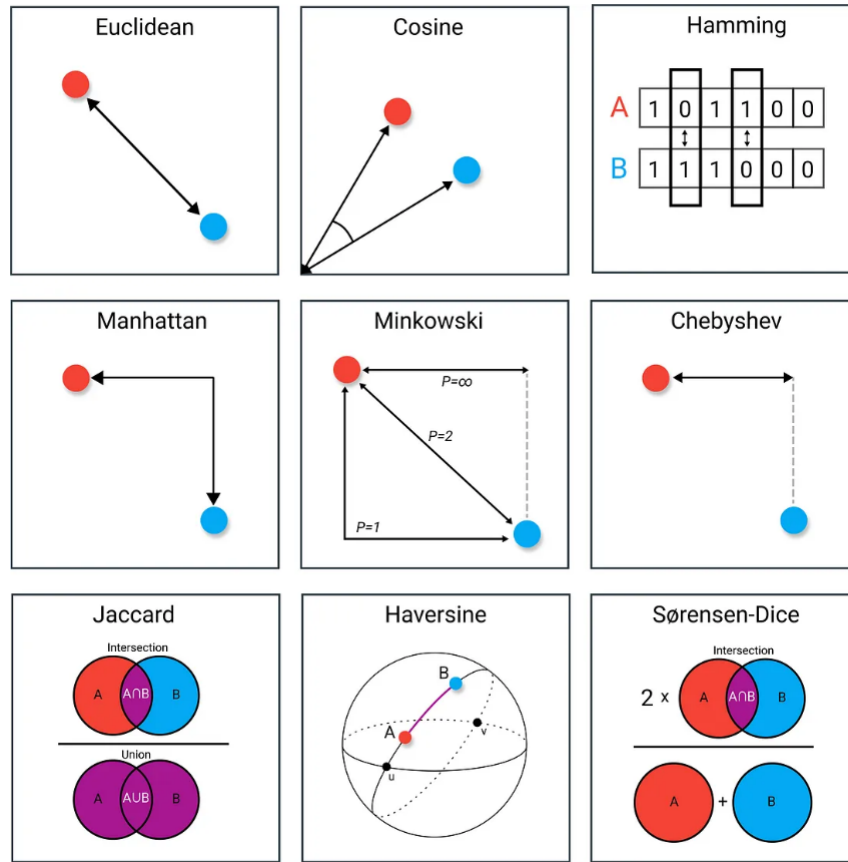


Figure 4: Some metrics commonly used in Machine Learning. source: Maarten Grootendorst

Metric spaces allow us to bring structure to sets like point clouds by giving us a precise way to talk about distances. This will be essential when we start building connections between points to uncover the hidden shape of data in Topological Data Analysis.

Now that we understand how to represent data as a **point cloud** and how to measure distances between points using **metric spaces**, the next step is to introduce some structure to these

scattered points. While point clouds give us a raw picture of where data points are located, they don't tell us how these points are connected or how they form meaningful shapes. To capture this hidden structure, we need mathematical tools that allow us to build shapes from points in a systematic way. This is where the concepts of **simplicial complexes** and **Homology** come into play.

3. Simplicial Complexes and Homology

In this section, we introduce the notions of **simplicial complexes** and **Homology**, which form the fundamental building blocks in Topological Data Analysis. Our primary focus will be understanding what simplicial complexes are and how they can be constructed from point clouds to reveal the underlying structure of data. We will then shift our focus to homology, which represents the focal point of persistent homology computation.

3.1. Simplicial Complexes

Simplicial complexes are mathematical structures that help us build and study the shapes or structures hidden within a set of scattered points (like a point cloud). They allow us to connect points, not just with lines, but also with higher-dimensional equivalents like triangles, tetrahedra, and beyond in a meaningful and systematic way. You can think of it as a flexible framework made up of simple pieces—points, edges, triangles, and their higher-dimensional counterparts—all glued together following specific rules.

Definition 6 (Abstract Simplicial Complex). *Let $\mathcal{V}$ be a non-empty set of points, called **vertices**. An **Abstract Simplicial Complex** of $\mathcal{K}$ over $\mathcal{V}$ is a collection of non-empty subsets of $\mathcal{V}$ that satisfies two simple rules:*

1. *Every single vertex $\{v\}$ with $v \in \mathcal{V}$ is in $\mathcal{K}$.*
2. *If a set of vertices σ belongs to $\mathcal{K}$, then all subsets of σ also belongs to $\mathcal{K}$.*

In simpler terms, a simplicial complex is formed by combining simplices in such a way that:

- All the vertices are part of the complex.
- Whenever you include a shape (like a triangle) $\{v_0, v_1, v_2\}$, you must also include all its edges $\{v_0, v_1\}, \{v_0, v_2\}, \{v_1, v_2\}$ and points $\{v_0\}, \{v_1\}, \{v_2\}$.

Each building block of the complex is called a simplex.

3.1.1. Construction of Simplicial Complexes

Once we understand what simplicial complexes are, the next question is: *How do we build them from data?* The construction typically begins with a **point cloud**—a set of points placed in a metric space where distances between points can be measured. In this context, each point in the cloud represents a **0-simplex**, which serves as the foundation for building higher-dimensional shapes. The next step is to create connections between these points to form higher-dimensional simplices:

- When two points are "close enough", we connect them with a line segment, forming a **1-simplex** (an edge).
- If three points are all pairwise connected, we can fill in the triangle between them to form a **2-simplex**.
- Similarly, four fully connected points form a **3-simplex** (a tetrahedron), and so on.

But how do we decide when points are "close enough" to be connected? This is where the idea of a **threshold distance** comes in.

Using distance to build connections. First, we calculate the distances between every pair of points in the cloud. However, these distances alone don't tell us which points should be connected. To make this decision, we introduce a threshold distance—a value that defines what we consider to be "close". If the distance between two points is less than or equal to this threshold, we draw an edge between them. As we increase the threshold distance, more points become connected, and higher-dimensional simplices begin to appear. Initially, the point cloud consists of many disconnected points, but as connections grow, the structure becomes richer and more informative.

Geometric representation. Mathematically, a d -dimensional simplex is represented as the **convex hull** of its $d + 1$ vertices in $\mathbb{R}^d$. This means it's the set of all weighted combinations of its vertices, where the weights are non-negative and sum to 1:

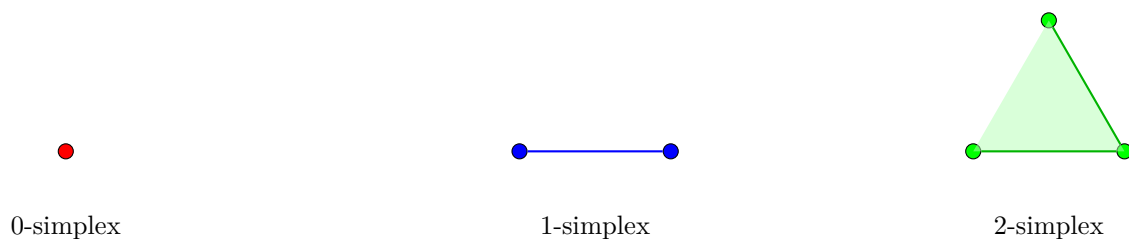
$$\left\{ \sum_{i=0}^d t_i v_i \mid t_i \geq 0, \sum_{i=0}^d t_i = 1 \right\},$$

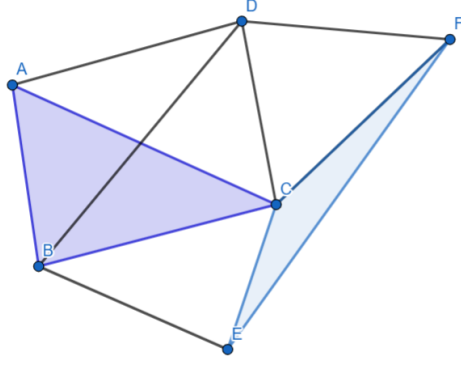
where v_i are the vertices and t_i are coefficients determining the position within the simplex.

Geometric Realization of a Simplicial Complex. Given a simplicial complex K , where K_0 is the set of vertices, and a mapping $\alpha : K_0 \rightarrow \mathbb{R}^N$ that places these vertices into space, the **geometric realization** of K , denoted by $|K|_\alpha$, is simply the union of all its simplices:

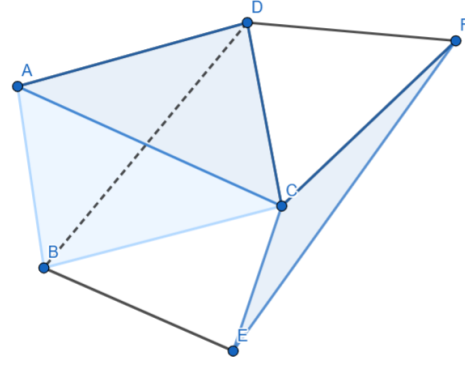
$$|K|_\alpha = \bigcup_{\sigma \in K} |\sigma|_\alpha.$$

This process transforms an abstract set of points and connections into a concrete geometric object that approximates the shape of the data. As we will see later, by varying the threshold distance step by step, we can create a sequence of simplicial complexes—a process called a **filtration**—which allows us to track how the shape of the data evolves across different scales.





(a) 2-Simplicial complex



(b) 3-simplicial complex

Figure 5: Example of simplicial complexes

The Figure 5 shows a geometric realization of a 2-simplicial complex representing the union of five 0-simplices, six 1-simplices, and two 2-simplices represented in the table below.

Table 1: k-dimensional simplices of the complex Figure 5

k-Simplices	<i>Complex 5a</i>	<i>Complex 5b</i>
0-simplices	$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}$	$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}$
1-simplices	$\{A,B\}, \{A,C\}, \{A,D\}, \{B,C\}, \{B,D\}, \{B,E\}, \{C,D\}, \{C,E\}, \{C,F\}, \{D,F\}, \{E,F\}$	$\{A,B\}, \{A,C\}, \{A,D\}, \{B,C\}, \{B,D\}, \{B,E\}, \{C,D\}, \{C,E\}, \{C,F\}, \{D,F\}, \{E,F\}$
2-simplices	$\{A,B,C\}, \{C,E,F\}$	$\{A,B,C\}, \{A,B,D\}, \{A,C,D\}, \{B,D,C\}, \{C,E,F\}$
3-simplices	NONE	$\{A,B,D,C\}$

As a simplicial complex might exhibit a high dimension, leading to high-dimensional simplices, the representation of lower-dimensional components of a d -simplex is given by its faces, which are l -simplices with $l < d$, where we are most interested in the $d - 1$ dimensional faces.

Definition 7 (Faces of a simplex). *A face τ of a simplex σ is a subset of σ such that all vertices of τ are also vertices of σ .*

A face of a simplex in a simplicial complex is a lower-dimensional simplex contained within it. Typically, we are particularly interested in the $(k - 1)$ -dimensional faces of a k -simplex, as they play a key role in defining the boundaries of the simplicial complex.

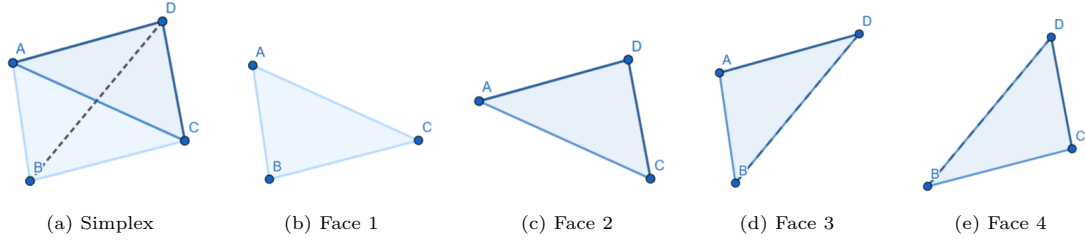


Figure 6: A Simplex with some of its faces

The Figure 6 shows in 6a a 3-simplex representing a tetrahedron with its two-dimensional faces, triangles 6c 6b 6d 6e.

3.2. Homology

Homology serves as the backbone of persistent homology, using chain complexes built from the simplices of a simplicial complex to capture the underlying structure of data across different scales. Homology reveals key topological features through the concept of chain groups and the calculation of **Betti numbers**. In this section, we will introduce the fundamental concepts of homology in a structured way, covering chain groups, boundary maps, chain complexes, homology groups and Betti Numbers.

3.2.1. Chain Groups, Boundary Maps, and Chain Complexes.

To understand how homology captures the shape of data, we first need to introduce a few key concepts: **chain groups**, **boundary maps**, and how they come together to form a **chain complex**.

Chain Groups. A chain group of a k -simplicial complex K over a field $\mathbb{F}$ is a vector space with its basis formed by the elements of K .

For each dimension k , we group together all the k -simplices of a simplicial complex K . These collections form what we call the k^{th} **chain group**, denoted $C_k(K)$. You can think of a chain group as a vector space where the basis elements are the simplices themselves, and we work over the field $\mathbb{Z}_2$ (where coefficients are either 0 or 1, indicating whether a simplex is included).

Definition 8 (Chain Group). *For each dimension $k \geq 0$, the vector space $C_k(K)$ over the field $\mathbb{Z}_2$, generated by the k -simplices of K , is called the k^{th} chain group of K [21].*

Example: Consider a simplicial complex made of two triangles sharing an edge (see Figure 6a). The 2nd chain group $C_2(K)$ consists of all linear combinations of these two triangles. Since there are two 2-simplices, $C_2(K)$ is isomorphic to $\mathbb{Z}_2^2$.

An element of a chain group, called a k -**chain**, is simply a sum of simplices:

$$\gamma = \sum_{\sigma \in K} \gamma_{\sigma} \sigma, \quad \text{where } \gamma_{\sigma} \in \mathbb{Z}_2.$$

Boundary Maps. Boundary maps provide a way to describe how higher-dimensional simplices are connected to their faces. This is done by "breaking down" a k -simplex into its $(k - 1)$ -dimensional faces.

Definition 9 (Boundary Map). *Given the k^{th} chain group $C_k(K)$, the **boundary map***

$$\delta_k^K : C_k(K) \longrightarrow C_{k-1}(K)$$

is a linear map that sends each k -simplex to a sum of its $(k - 1)$ -dimensional faces.

For a simplex $\sigma = [v_0, v_1, \dots, v_k]$, the boundary map is defined in general as:

$$\delta_k^K(\sigma) = \sum_{i=0}^k (-1)^i \sigma_{-i},$$

where σ_{-i} denotes the i^{th} face of σ . However when working over the field $\mathbb{Z}/2\mathbb{Z} = \{0, 1\}$, signs are suppressed because $-1 \cong 1 \pmod{2}$. This means each face appears with coefficients 0 or 1. In this case, all faces equally contribute to the formation of the complex, and a face will be cancelled out if it appears twice as $1 + 1 = 0 \pmod{2}$. Then the boundary map is simplified to:

$$\delta_k^K(\sigma) = \sum_{i=0}^k \sigma_{-i},$$

Since boundary maps are linear, they act on chains by distributing over sums:

$$\delta_k^K(\gamma) = \sum_{\sigma} \gamma_{\sigma} \delta_k^K(\sigma).$$

Example 3 (Boundary of a 1-chain). *Let's $C = [v_0, v_1] + [v_1, v_2] + [v_2, v_3] + [v_3, v_4] + [v_4, v_5] + [v_5, v_0]$ be a 1-chain*

$$\begin{aligned} \delta_1(C) &= \delta_1([v_0, v_1]) + \delta_1([v_1, v_2]) + \delta_1([v_2, v_3]) + \delta_1([v_3, v_4]) + \delta_1([v_4, v_5]) + \delta_1([v_5, v_0]) \\ &= [v_0] + [v_1] + [v_1] + [v_2] + [v_2] + [v_3] + [v_3] + [v_4] + [v_4] + [v_5] + [v_5] + [v_0] \\ &= [v_0] + [v_0] + [v_1] + [v_1] + [v_2] + [v_2] + [v_3] + [v_3] + [v_4] + [v_4] + [v_5] + [v_5] \\ &= 2[v_0] + 2[v_1] + 2[v_2] + 2[v_3] + 2[v_4] + 2[v_5] \\ &= 0 + 0 + 0 + 0 + 0 + 0 = 0 \end{aligned}$$

Definition 10 (Cycle). *Let's C be a k -chain over a field $\mathbb{Z}/2\mathbb{Z}$. A k -cycles is a k -chain with zero boundary. Meaning that, C is a k -cycle if and only if*

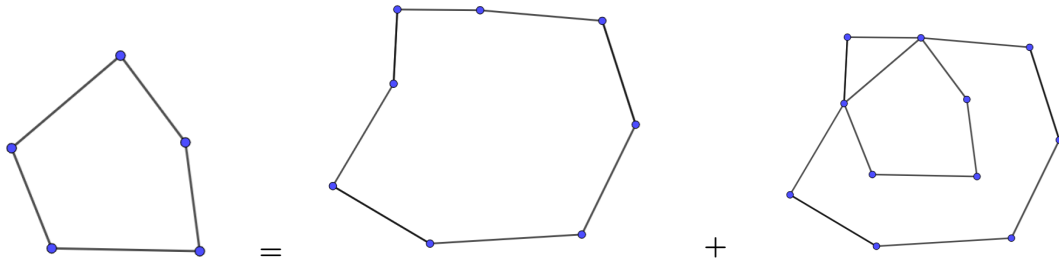
$$\delta_k(C) = 0$$

.

Example 3 demonstrates that $\delta_1(C) = 0$, which implies that C is a 1-cycle.

Two 1-cycles C and C' are said to be homologous, or equivalent in homology, (or the same) if their sum lies in the image of the boundary map, that is, if

$$C + C' \in \text{im}(\delta)$$



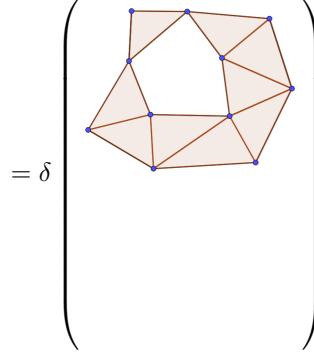


Figure 8: Example of boundary of an object

Chain Complexes. At a high level, a **chain complex** is a mathematical structure that helps us organize and analyze the different building blocks (simplices) of a simplicial complex across various dimensions. It provides a systematic way to track how these simplices connect to each other.

Now that we have chain groups and boundary maps, we can link them to form a **chain complex**—a sequence of vector spaces connected by boundary maps.

Definition 11 (Chain Complex). A **chain complex** is a sequence of chain groups connected by linear maps (boundary maps) such that applying two boundary maps in a row always gives zero:

$$d_{k-1} \circ d_k = 0 \quad \text{for all } k.$$

In the case of simplicial complexes, this sequence looks like:

$$0 \longrightarrow C_k(K) \xrightarrow{\delta_k} C_{k-1}(K) \xrightarrow{\delta_{k-1}} \cdots \xrightarrow{\delta_1} C_0(K) \longrightarrow 0.$$

Example 4 (Simplicial Chain Complex). For a 3-dimensional simplicial complex K :

- $C_3(K)$: generated by tetrahedrons (3-simplices),
- $C_2(K)$: generated by triangles (2-simplices),
- $C_1(K)$: generated by edges (1-simplices),
- $C_0(K)$: generated by vertices (0-simplices).

The chain complex is:

$$0 \longrightarrow C_3(K) \xrightarrow{\delta_3} C_2(K) \xrightarrow{\delta_2} C_1(K) \xrightarrow{\delta_1} C_0(K) \longrightarrow 0.$$

From Chains to Homology. With the chain complex in place, we can now define important concepts like:

- The **kernel** of a boundary map, which identifies **cycles** (closed structures with no boundary).
- The **image** of a boundary map, which corresponds to **boundaries** (those chains that are themselves boundaries of higher-dimensional simplices).

The difference between cycles and boundaries—formally, the kernel modulo the image—gives us the **homology groups**, which capture the essential topological features of the space. These features are quantified using **Betti numbers**, telling us how many connected components, loops, or voids exist at each dimension.

3.2.2. Homology Groups and Betti Numbers

In topology, one of the key challenges is understanding the essential features of a shape—those characteristics that remain unchanged even if the object is stretched, bent, or twisted. Capturing and counting these features is crucial for unveiling the intrinsic structure of the data. This is where the concepts of **homology** and **betti numbers** come into play.

Homology. Homology provides a systematic way to detect and classify these fundamental features, known as **topological invariants**. These include:

- **Connected components** (isolated pieces),
- **Loops or cycles** (like holes in a doughnut),
- **Voids** (empty spaces enclosed in higher dimensions),
- And other features in higher-dimensional spaces.

The key idea behind homology is to capture these features using algebra. This is done by constructing what are called **homology groups**.

A natural question that may come to mind is **Why Homology?**

Imagine deforming a rubber band into different shapes—it might stretch or twist, but as long as you don't cut it or glue parts together, it remains a loop. Homology helps us formalize this intuition. It allows us to identify features that persist through continuous deformations, while recognizing that operations like cutting or gluing change the fundamental structure.

To study these unchanging properties, we focus on detecting "holes" of different dimensions within a space. The mathematical tool that captures this information is the **homology group**.

Definition 12 (Homology Group). *Given a simplicial complex K , the k^{th} homology group, denoted $H_k(K)$, is defined as:*

$$H_k(K) = \ker(\delta_k) / \text{im}(\delta_{k+1}),$$

The homology group captures the k -dimensional features that are cycles but not boundaries, meaning they represent "holes" in dimension k .

The dimension of the homology group, denoted

$$\dim(H_k(K)) = \dim \ker(\delta_k) - \dim \text{im}(\delta_{k+1})$$

represents the number of independent k -dimensional cycles which are not boundaries of a $(k + 1)$ -dimensional simplices. Breaking down the formula:

- $\ker(\delta_k)$ (the kernel) counts all k -dimensional cycles.
- $\text{im}(\delta_{k+1})$ (the image) counts all k -dimensional cycles that bound $(k + 1)$ -dimensional simplices.
- $\ker(\delta_k) / \text{im}(\delta_{k+1})$ counts all k -dimensional cycles that do not bound $(k + 1)$ -dimensional simplices.

This dimension is called k^{th} Betti number, denoted by β_k , and it summarizes the number of k -dimensional holes in the space.

Betti Numbers. Betti numbers count the topological features and provide a numerical summary of how many features exist in each dimension.

The k^{th} **Betti number**, denoted β_k , counts the number of independent k -dimensional holes:

- β_0 : Number of connected components,
- β_1 : Number of loops or 1-dimensional holes,
- β_2 : Number of voids or enclosed surfaces,
- and so on for higher dimensions.

For example:

- A single circle has $\beta_0 = 1$ (it's connected) and $\beta_1 = 1$ (it has one loop).
- A sphere has $\beta_0 = 1$, $\beta_1 = 0$ (no loops), and $\beta_2 = 1$ (it encloses a void).

Why this matters. By calculating homology groups and Betti numbers, we gain powerful insight into the shape of data or objects—insights that remain valid even when the object is deformed. This is the foundation of many techniques in Topological Data Analysis, where understanding the "shape" of data reveals patterns that traditional methods might miss.

In the next sections, we will see how these concepts are applied in practice, particularly through **Persistent Homology**, where we track how these topological features appear and disappear as we change the scale of observation.

Table 2: Betti numbers for common shapes

Shape	β_0	β_1	β_2
Circle (S^1)	1	1	0
Sphere (S^2)	1	0	1
Torus	1	2	1
Two Disjoint Circles	2	2	0

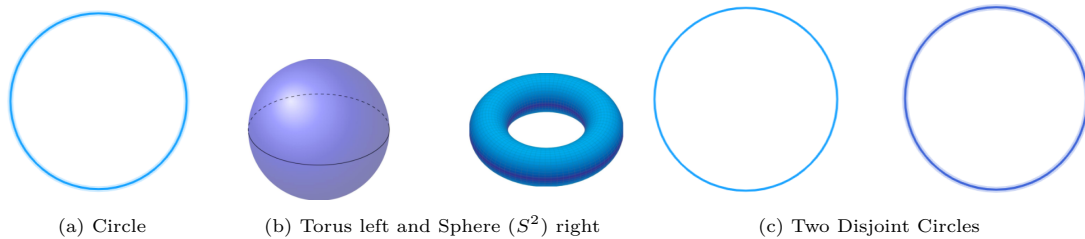


Figure 9: Visual Representation of Common Shapes and Their Topological Features

4. Persistent Homology

Persistent homology (PH), a fundamental tool in *Topological Data Analysis* (TDA), emerged from advancements in applied algebraic topology and computational geometry [9, 30]. It was developed to address limitations of traditional machine learning models, particularly their inability to effectively handle incomplete or high-dimensional data, and to capture the *global geometric structure* within such datasets [4]. In particular, PH is very strong in dealing with noisy data, and we attribute this to the stability theorem [6, 7, 24], which guarantees that small perturbations of the input data will lead to a small change in the resulting persistent diagram. To measure the topological differences between the persistent diagrams, distances such as the bottleneck distance and the Wasserstein distance are used.

By tracking topological features—such as connected components, loops, holes, and voids—that persist across multiple scales, PH reveals intrinsic patterns often hidden from classical statistical or geometric approaches [12]. This robustness makes PH a valuable method in diverse fields, from biology to finance.

Persistent homology operates by constructing a nested sequence of simplicial complexes, known as a **filtration**, and computing homology at each stage. The result is a vector space whose basis consists of *topological invariants*, providing a multi-scale summary of the data’s shape [8]. This process captures features that are stable under perturbations, offering insights into the underlying structure of complex datasets.

4.1. Filtration and the birth/death of topological features

This section explores how a filtration is constructed from data and how topological features emerge and disappear as the scale parameter varies.

4.1.1. Filtration

A **filtration** refers to a sequence of nested simplicial complexes built by gradually increasing a scale parameter ϵ . For point cloud data, common filtrations include the *Vietoris–Rips* and *Čech* complexes [23]. Formally, given a point cloud X , a filtration can be represented as:

$$K_0 \subseteq K_1 \subseteq K_2 \subseteq \cdots \subseteq K_n$$

where each K_i includes simplices based on proximity criteria defined by ϵ . As ϵ increases, more simplices are added, enabling the detection of evolving topological features.

4.1.2. birth/death of topological features

During the filtration process, topological features such as connected components (H_0), loops (H_1), and voids (H_2) appear (*birth*) and eventually disappear or merge (*death*). The interval between birth and death represents the **persistence** of each feature. Long-persisting features are typically considered meaningful, while short-lived ones are often attributed to noise.

Thanks to a fundamental result in algebraic topology referred to as the structure theorem for persistence modules [6, 24], it’s true that the persistent homology of a finite filtered simplicial complex can be completely characterized by a multiset of intervals. These intervals are what we visualize as barcodes, with each bar representing the lifespan of a topological feature across the filtration.

4.2. Barcodes and Persistence diagrams

To visualize persistent homology, two primary tools are used: **barcodes** and **persistence diagrams** [8].

Barcodes represent each topological feature as a horizontal line segment, where the endpoints correspond to its birth and death scales. Longer bars indicate more persistent—and thus more significant—features.

Persistence diagrams plot features as points in a two-dimensional plane, with birth on the x-axis and death on the y-axis. Features farther from the diagonal represent higher persistence.

These visual tools not only aid interpretation but also enable integration into machine learning pipelines through vectorization techniques like *persistence landscapes* and *persistence images* [1, 3].

Persistent homology has demonstrated effectiveness in various applications, including:

1. **Biology:** Protein structure and folding analysis [18].
2. **Finance:** Identifying patterns in stock market behavior [13].
3. **Neuroscience:** Studying brain network connectivity [14].

Its ability to capture non-linear, multi-scale structures while remaining robust to noise makes PH an essential tool in modern data analysis.

5. Application: DNA Structure

To illustrate the practical power of **Persistent Homology** (PH), this section presents an example of analyzing a protein’s 3D structure. Proteins, due to their complex folding patterns, are ideal candidates for topological analysis, as their shapes contain rich structural information that traditional geometric tools might overlook.

Step 1: Extracting Protein Data as a Point Cloud. Proteins are made up of chains of amino acids, and their 3D structure is often represented by tracing the positions of specific atoms—most commonly the **alpha-carbon ($C\alpha$)** atoms along the protein backbone. These (x, y, z) coordinates form a natural **point cloud**, which can then be analyzed using topological methods.

You can obtain protein 3D structures from several publicly available databases:

- **AlphaFold Database** - AI-predicted protein structures.
- **KnotProt** - A database focused on proteins with knots and entanglements.
- **Protein Data Bank (PDB)** – A comprehensive repository of experimentally determined protein structures.

Once downloaded, these structures (typically in ".pdb" format) provide the coordinates needed for topological analysis.

Step 2: Computing the Filtration. With the point cloud in hand, the next step is to build a **filtration**—a sequence of simplicial complexes generated by connecting points based on a growing distance threshold. This process tracks how topological features (like loops and voids) appear and disappear as we change the scale.

For this, we recommend using specialized libraries such as:

- **Ripser** — A fast C++/Python library for computing Vietoris–Rips persistent homology.
- **GUDHI** — A robust Python and C++ library offering tools for simplicial complexes, filtrations, and persistent homology.

Installation:

```
pip install ripser
pip install gudhi
```

These libraries will compute when each topological feature is **born** and when it **dies** during the filtration process. We will use the **GUDHI** library for our computations.

Step 3: Visualizing Topological Features. The results of persistent homology are typically visualized using:

- **Persistence Barcodes:** Each feature is represented by a horizontal bar, where the length indicates how long it persists.
- **Persistence Diagrams:** Each feature is plotted as a point in 2D space, where the x-axis represents its birth time and the y-axis its death time.

In a persistence diagram, the diagonal line $y = x$ represents features that appear and disappear almost immediately—these are usually considered **noise**. Features farther from this line are interpreted as meaningful **topological invariants**, revealing the true structural patterns within the protein.

Step 4: Interpreting Betti Numbers. The topological features detected are quantified using **Betti numbers**:

- B_0 : Number of connected components (0-dimensional holes).
- B_1 : Number of loops or cycles (1-dimensional holes).
- B_2 : Number of voids (2-dimensional holes).

These numbers provide a summary of the protein's topological complexity at different dimensions.

Step 5: Computing Homology Generators. While Betti numbers tell us *how many* features exist, it's also useful to identify *where* these features are located within the structure. This requires **computing homology generators**. For this task, you can use:

- **Eirene.jl** — A Julia package designed for advanced persistent homology computations.
- **GUDHI** — In addition to filtrations, GUDHI also supports homology generator extraction.

Eirene.jl Installation:

```
using Pkg
Pkg.add("Eirene")
```

In our case, we will skip the last step as our focus is limited to the computation of Betti numbers.

Conclusion. By applying persistent homology to protein structures, we gain insights into the global geometric and topological properties of biomolecules—information that can be crucial in understanding protein folding, stability, and function. This method is robust to noise and dimensionality, making it a valuable tool in computational biology.

In the next section, we will walk through a complete Python implementation of this process using GUDHI and Ripser.

5.1. Examples of persistent homology to analyze 3-1m-Supercoiled.

This paragraph will move us step by step through the computation of topological invariants of the 3-1m-supercoiled DNA molecules sequence. Since the structure of the protein sequence is known, the 3D structure is downloaded from the AlphaFold database[29], where we extracted the xyz coordinates of carbon-alpha atoms. The coordinates will then be used to construct simplicial complexes that will be the key element of homology group computation.

1. xyz coordinates of carbon-alpha atoms of 3-1m-supercoiled DNA molecules.

The analysis start with a 3-1m-supercoiled DNA molecules sequence represented as x, y, z coordinates in $\mathbb{R}^3$

Row	1	2	3
1	9.09534	-22.3605	-2.41842
2	11.2318	-23.7001	-4.19976
3	11.183	-26.6309	-4.48222
4	9.45528	-28.8571	-3.66466
5	7.2711	-31.1725	-3.84429
6	5.13819	-33.3921	-4.02771
7	4.80984	-36.4388	-3.72263
8	4.68212	-39.5552	-4.3185
9	5.97231	-42.0341	-3.16073
10	6.09297	-43.1175	-0.272976
11	4.10824	-43.558	2.06122
12	1.28592	-43.4943	3.25026

Figure 10: x, y, z coordinates of 3-1m-Supercoiled

The figure 10 is a table with 3 columns and 111 rows. Where the columns are respectively the x, y , and z coordinates, and the number of rows represents the number of points used to represent the 3D structure of the 3-1m-Supercoiled DNA structure. Having such coordinates, the best way to give a sense of its hidden structure is to visualize them.

2. Point Cloud

Having the x, y, z coordinates of DNA structure, we can visualize the point cloud representing those coordinates using a scatter plot as shown by the figure 11

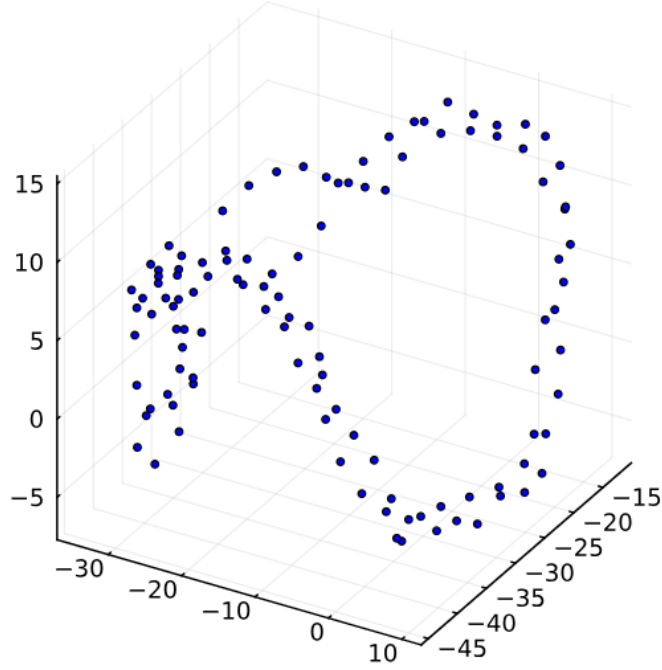


Figure 11: Point cloud of the 3-1m-Supercoiled.

From the figure 11, we can see each data point with its corresponding x, y, z coordinates, leading us to the notion of neighbourhood. We want to measure how data points are close or far from each other, and for that, we will start by computing pairwise distances between them, and after that, we will define our understanding of closeness by setting a threshold distance and make a comparison with the computed distances. Two points will be said to be close if the distance between them is less than or equal to the threshold distance, and then, an edge will be constructed between them, and the corresponding edge is called a 1-simplex.

3. Pairwise distances

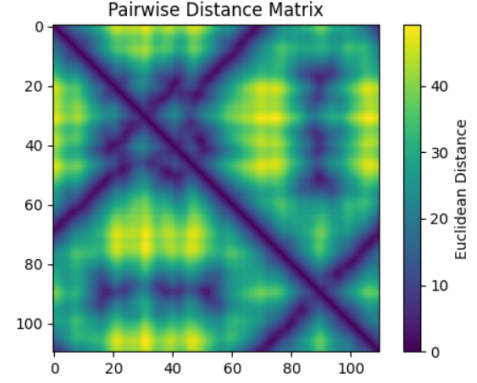
The pairwise distances are the different distances given between each pair of points. So the distance matrix obtained is a 111×111 matrix where rows and columns are the same, representing the different data points, the the values are the corresponding Euclidean distances obtained.

From the figure 12a, we can see that the distance between a point and itself is zero as defined by a distance metric.

Using the distance matrix, we can start building simplicial complexes by adding edges each time that points are close, meaning when pairwise distances are less than or equal to the threshold distance.

Row	1	2	3	4	5	6
1	0	3.08741907819466	5.182075823372809	6.62483830004015	9.111108650111463	11.829837234155013
2	3.08741907819466	0	2.9447841570478674	5.480602550851503	8.464647964971737	11.440740085897142
3	5.182075823372809	2.9447841570478674	0	2.9341737281622874	6.027938856267569	9.08075617535236
4	6.62483830004015	5.480602550851503	2.9341737281622874	0	3.1881007864162923	6.271785740170035
5	9.111108650111463	8.464647964971737	6.027938856267569	3.1881007864162923	0	3.08375941417301
6	11.829837234155013	11.440740085897142	9.08075617535236	6.271785740170035	3.08375941417301	0
7	14.773794531673307	14.27388177646302	11.72130742552639	8.89188665452792	5.81432724797996	3.0794915844827413
8	17.83347580231424	17.15507514702282	14.46809508948085	11.72385586467335	8.786262356782158	6.186789173634457
9	19.93376225093992	19.101763646349518	16.314264233469006	13.63865487750421	10.902013147783716	8.72334093859951
10	21.08246201405415	20.466113711855694	17.76046308455059	15.038871535190273	12.52299816234593	10.468669324186154
11	22.23223666372547	22.063893047383	19.478078716885797	16.6381211225238	14.08116976638354	11.854594921955125
12	23.23270272953193	23.371643639744737	21.050395041442167	18.13290387716427	15.426057019812072	12.03103641755302

(a) pairwise distances data array



(b) Heatmap of pairwise distances

Figure 12: Pairwise distances illustrating proximity between residus

Vietoris-Rips filtration

The next step is to compute the filtration using Vietoris-Rips filtration as illustrated by the figure [13,14] and showing that from a radius $r = 6.5$ we start seeing a persistent hole that represents a one-dimensional hole.

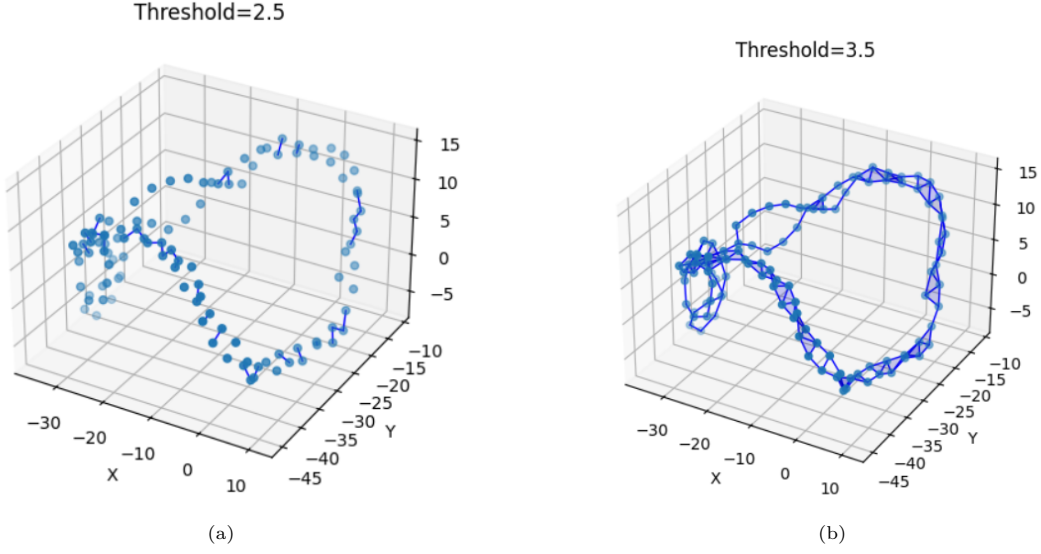


Figure 13: Initial Stages of Vietoris-Rips Filtration: Early Edge Formation and Emerging Loops.

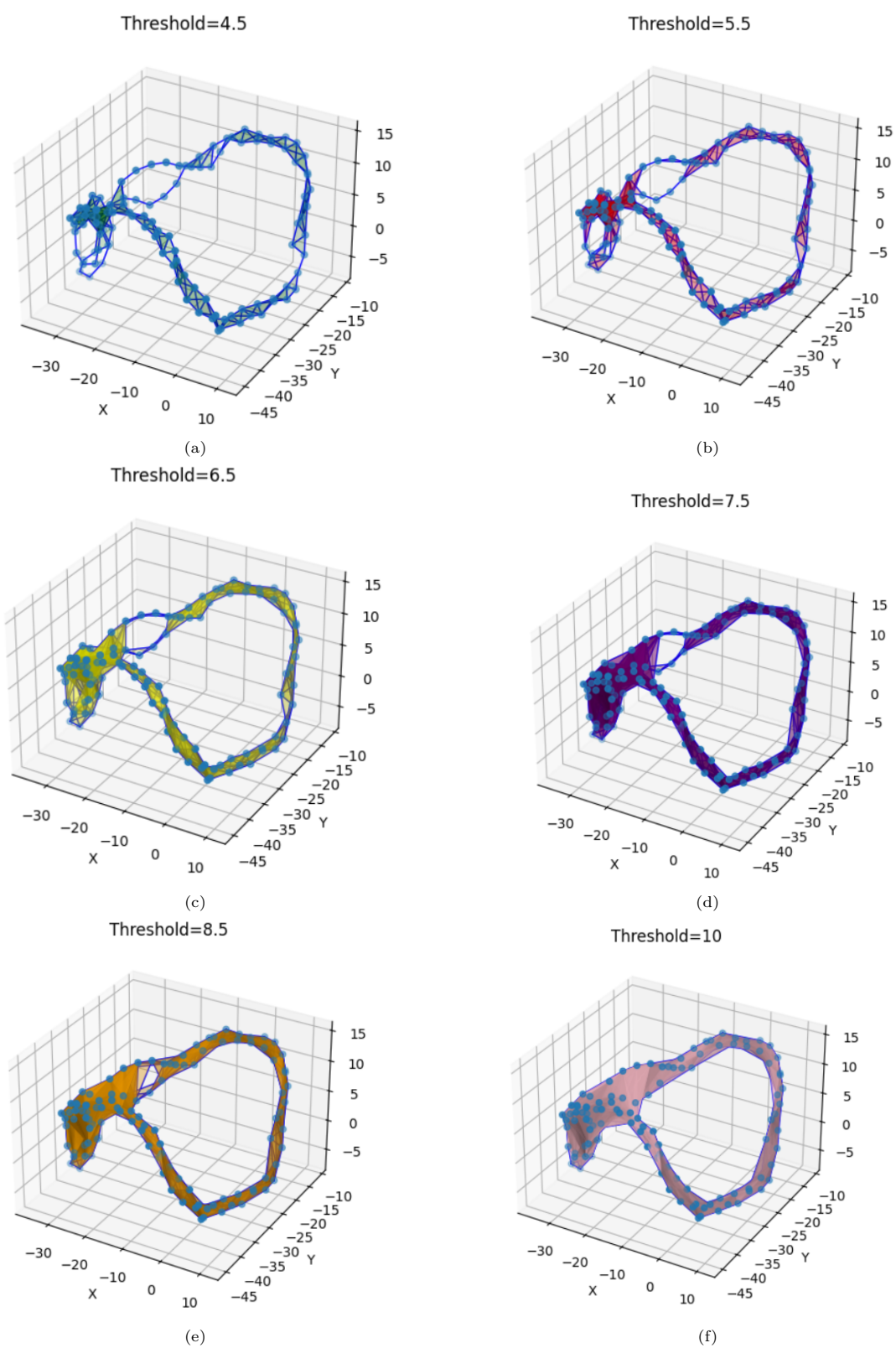


Figure 14: Vietoris-Rips Filtration Revealing the Birth and Death of 1-Dimensional Homological Features.

4. Persistent diagram

The persistent diagram of the filtration represented by simplicial complexes at different scales can be visualised using the persistent barcode or persistent diagram. The figure 15 shows on the left a persistent diagram and persistent barcode representing different homology classes, where each coordinate is defined by the birth time and the death time of each topological feature.

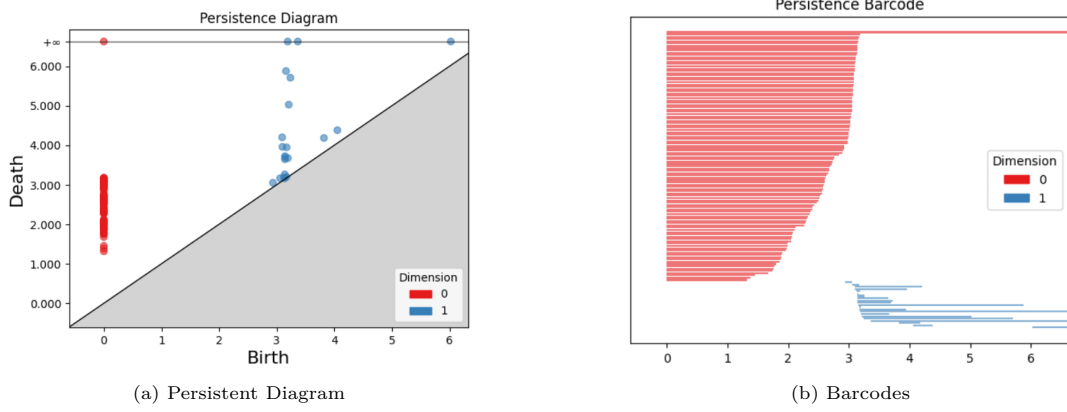


Figure 15: Persistence Diagram (left) and Persistence Barcode (right) showing topological features across scales computed using Vietoris-Rips Filtration. H_0 represents connected components, H_1 represents loops or cycles, and H_2 would represent voids (none detected here).

The persistence diagram shows a large number of long-lived H_0 features (red), indicating that many connected components persist across scales before merging. A few H_1 features (blue) appear but have relatively short lifespans, suggesting minor or less significant loops within the data.

Overall, the barcode confirms that the dataset is characterized by well-separated clusters or components with limited cyclic structure. This topological signature implies that machine learning algorithms focusing on connectivity or clustering could benefit from incorporating these persistent H_0 features, while the limited H_1 features may play a lesser role in downstream tasks.

5. Betti numbers computation

Table 3: Betti Numbers for 3-1m-Supercoiled

β_0	β_1	β_2
1	3	0

The Betti numbers presented in the above table were computed using a Python-based pipeline leveraging the **Ripser** library for topological data analysis. The numbers describe the topological structure of the 3-1m-Supercoiled DNA molecules. Specifically, $\beta_0 = 1$ indicates that the structure is composed of a single connected component. The value $\beta_1 = 3$ reveals the presence of three significant loops or cycles within the protein configuration, which may correspond to biologically relevant structural features such as twists or knots. Finally, $\beta_2 = 0$ confirms the absence of voids

or enclosed cavities in the data, which is expected given the nature of the spatial representation of the protein.

The complete Python code can be found on this GitHub repository.

6. Conclusion and Future Directions

This final section provides a systematic overview of the key points discussed and outlines potential future directions.

6.1. Conclusion

Persistent homology (PH) has been demonstrated to effectively complement traditional machine learning techniques by systematically uncovering the global structure of data across multiple scales. This is achieved through filtration functions, which capture topological features such as loops and voids. Furthermore, PH primarily operates on point cloud data or data that has been sampled into a point cloud, leveraging Euclidean distance to define proximity between points and construct a simplicial complex.

6.2. Future Directions

An interesting future direction could be exploring how persistent homology can be integrated into neural network architectures. Since both approaches involve hierarchical learning, combining them may provide deeper insights into the representations learned at each hidden layer, potentially improving interpretability and model understanding.

Another promising yet challenging future direction is the integration of Topological Data Analysis (TDA) with Graph Neural Networks (GNNs). While recent efforts have explored this intersection, existing results have not consistently demonstrated significant improvements in predictive performance over existing models [25, 28].

References

- [1] Henry Adams and Tamal K. et al. Emerson. Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.
- [2] Manu Aggarwal and Vipul Periwal. Tight basis cycle representatives for persistent homology of large biological data sets. *PLOS Computational Biology*, 19(5):e1010341, 2023.
- [3] Peter Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16(1):77–102, 2015.
- [4] Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46(2): 255–308, 2009.
- [5] Gunnar Carlsson, Rick Jardine, Dmitry Feichtner-Kozlov, Dmitriy Morozov, Frédéric Chazal, Vin de Silva, Brittany Fasy, Jesse Johnson, Matt Kahle, Gilad Lerman, et al. Topological data analysis and machine learning theory. In *Proc. BIRS Workshop*, pages 1–11, 2012.
- [6] Frédéric Chazal, Vin De Silva, Marc Glisse, and Steve Oudot. *The structure and stability of persistence modules*, volume 10. Springer, 2016.
- [7] David Cohen-Steiner, Herbert Edelsbrunner, and John Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.

- [8] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.
- [9] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002.
- [10] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231, 1996.
- [11] Colleen M Farrelly and Yaé Ulrich Gaba. *The Shape of Data: Geometry-Based Machine Learning and Data Analysis in R*. No Starch Press, 2023.
- [12] Robert Ghrist. Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1):61–75, 2008.
- [13] Marian Gidea and Yuri Katz. Topological data analysis of financial time series: Landscapes of crashes. *Physica A: Statistical Mechanics and its Applications*, 491:820–834, 2018.
- [14] Chad Giusti, Robert Ghrist, and Danielle S. Bassett. Two’s company, three (or more) is a simplex: Algebraic-topological tools for understanding higher-order structure in neural data. *Journal of Computational Neuroscience*, 41(1):1–14, 2016.
- [15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [16] Florian Hahne, Wolfgang Huber, Robert Gentleman, Seth Falcon, R Gentleman, and VJ Carey. Unsupervised machine learning. *Bioconductor case studies*, pages 137–157, 2008.
- [17] Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.
- [18] Peter M. Kasson. Topological data analysis of protein folding dynamics. *Current Opinion in Structural Biology*, 60:138–144, 2020.
- [19] Iqbal Muhammad and Zhu Yan. Supervised machine learning approaches: A survey. *ICTACT Journal on Soft Computing*, 5(3), 2015.
- [20] Samreen Naeem, Aqib Ali, Sania Anam, and Muhammad Munawar Ahmed. An unsupervised machine learning algorithms: Comprehensive review. *International Journal of Computing and Digital Systems*, 2023.
- [21] Vidit Nanda. Computational algebraic topology lecture notes. URL: <https://people.maths.ox.ac.uk/nanda/cat/TDANotes.pdf>, 2021.
- [22] Vladimir Nasteski. An overview of the supervised machine learning methods. *Horizons. b*, 4 (51-62):56, 2017.
- [23] Nina Otter, Mason A Porter, Ulrike Tillmann, Peter Grindrod, and Heather A Harrington. A roadmap for the computation of persistent homology. *EPJ Data Science*, 6(1):17, 2017.
- [24] Steve Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis*, volume 209 of *Mathematical Surveys and Monographs*. American Mathematical Society, 2015.

- [25] Phu Pham, Quang-Thinh Bui, Ngoc Thanh Nguyen, Robert Kozma, Philip S Yu, and Bay Vo. Topological data analysis in graph neural networks: surveys and perspectives. *IEEE Transactions on Neural Networks and Learning Systems*, 2025.
- [26] Chi Seng Pun, Kelin Xia, and Si Xian Lee. Persistent-homology-based machine learning and its applications – a survey, 2018. URL <https://arxiv.org/abs/1811.00252>.
- [27] Yohai Reani and Omer Bobrowski. Cycle registration in persistent homology with applications in topological bootstrap. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5579–5593, 2022.
- [28] Funmilola Mary Taiwo, Umar Islambekov, and Cuneyt Gurcan Akcora. Explaining the power of topological data analysis in graph machine learning. *arXiv preprint arXiv:2401.04250*, 2024.
- [29] Mihaly Varadi, Stephen Anyango, Mandar Deshpande, Sreenath Nair, Cindy Natassia, Galabina Yordanova, David Yuan, Oana Stroe, Gemma Wood, Agata Laydon, et al. Alphafold protein structure database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic acids research*, 50(D1):D439–D444, 2022.
- [30] Afra Zomorodian and Gunnar Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005.